

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

**Дисциплина:
«Программирование»**

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Указатели и динамическая память»

вариант - 5_19

Выполнил:

студент гр. N3151

Агакарян Арам Арманович



(подпись)

Проверил:

инженер ФБИТ

Волков Александр Григорьевич

(отметка о выполнении)

(подпись)

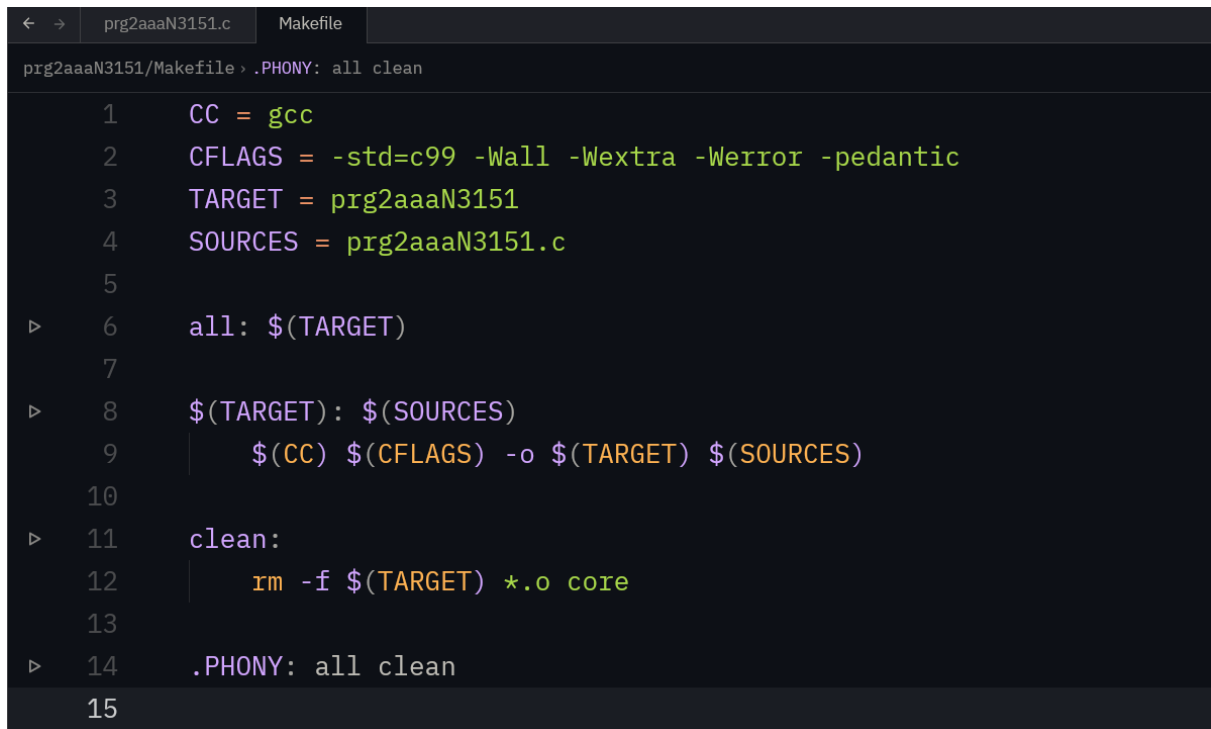
Санкт-Петербург
2025 г.

Способ представления матрицы - VLA

Задание: существовать в матрице циклический сдвиг на T элементов влево каждой диагонали с нечетным порядком и на T элементов вправо каждой диагонали с четным порядком.

Тип элементов: double

Makefile



```
prg2aaaN3151/Makefile > .PHONY: all clean

1  CC = gcc
2  CFLAGS = -std=c99 -Wall -Wextra -Werror -pedantic
3  TARGET = prg2aaaN3151
4  SOURCES = prg2aaaN3151.c
5
6  all: $(TARGET)
7
8  $(TARGET): $(SOURCES)
9      $(CC) $(CFLAGS) -o $(TARGET) $(SOURCES)
10
11 clean:
12     rm -f $(TARGET) *.o core
13
14 .PHONY: all clean
15
```

```
CC = gcc
CFLAGS = -std=c99 -Wall -Wextra -Werror -pedantic
TARGET = prg2aaaN3151
SOURCES = prg2aaaN3151.c
```

```
all: $(TARGET)
```

```
$(TARGET): $(SOURCES)
$(CC) $(CFLAGS) -o $(TARGET) $(SOURCES)
```

```
clean:
rm -f $(TARGET) *.o core
```

```
.PHONY: all clean
```

примеры работы программ

```
~/Desktop/prg2aaaN3151 via C v13.3.0-gcc
```

```
16:26:06 $ ./prg2aaaN3151 -m 3 3
```

Введите элементы матрицы 3x3:

1
2
3
4
5
6
7
8
9

Введите T: 11

Исходная матрица:

1.000	2.000	3.000
4.000	5.000	6.000
7.000	8.000	9.000

Результат:

5.000	6.000	3.000
8.000	9.000	2.000
7.000	4.000	1.000

```
~/Desktop/prg2aaaN3151 via C v13.3.0-gcc took 10s
```

```
16:26:17 $ ./prg2aaaN3151 4 5
```

Введите T: 3

Исходная матрица:

22.960	46.260	94.530	19.600	91.920
93.230	10.960	69.270	25.480	83.880
53.130	50.190	10.870	23.140	2.380
80.360	38.040	51.780	10.050	92.420

Результат:

10.960	92.420	94.530	83.880	91.920
93.230	10.870	46.260	25.480	19.600
38.040	50.190	10.050	69.270	2.380
80.360	53.130	51.780	22.960	23.140

```
~/Desktop/prg2aaaN3151 via C v13.3.0-gcc took 7s
```

```
16:28:23 $ LAB2DEBUG=1 ./prg2aaaN3151 4 3
```

Введите T: 1

Исходная матрица:

49.920	63.240	63.950
12.910	89.210	19.810
74.930	55.290	10.190
40.710	3.670	8.590

[DEBUG] Диагональ V=-2, длина=2, сдвиг=1, направление=вправо (чётная)

[DEBUG] Диагональ V=-1, длина=3, сдвиг=1, направление=влево (нечётная)

[DEBUG] Диагональ V=0, длина=3, сдвиг=1, направление=вправо (чётная)

[DEBUG] Диагональ V=1, длина=2, сдвиг=1, направление=влево (нечётная)

Результат:

10.190	19.810	63.950
55.290	49.920	63.240
3.670	8.590	89.210
40.710	74.930	12.910

```
~/Desktop/prg2aaaN3151 via C v13.3.0-gcc
```

```
04:01:43 $ ./prg2aaaN3151 0 5
```

Ошибка: некорректные размеры матрицы

```
~/Desktop/prg2aaaN3151 via C v13.3.0-gcc
```

```
04:01:56 $ ./prg2aaaN3151 -k 4 5
```

Ошибка: опция '-k' не поддерживается

исходный текст программы с комментариями

```
#include <errno.h> #include <stdio.h> #include <stdlib.h> #include <string.h> #include <time.h>

/* чтение целого числа из строки */ int parse_int(const char *s, int *x) { char *endptr; errno = 0; long val
= strtol(s, &endptr, 10);

if (endptr == s || *endptr != '\0' || errno != 0) return 0;

if (val <= 0 || val > 1000) return 0;

*x = (int)val; return 1; }

/* чтение числа double */ int read_double(double *x) { char buf[256]; char *endptr;

if (!fgets(buf, sizeof(buf), stdin)) return 0;

errno = 0; double val = strtod(buf, &endptr);

if (endptr == buf || errno != 0) return 0;

*x = val; return 1; }

/* вывод матрицы */ void print_matrix(int n, int m, double a[n][m]) { for (int i = 0; i < n; i++) { for (int j =
0; j < m; j++) printf("%8.3f ", a[i][j]); printf("\n"); } }

/* циклический сдвиг диагоналей */ void shift_diagonals(int n, int m, double a[n][m], int T) { int debug
= getenv("LAB2DEBUG") != NULL;

/* перебор всех диагоналей */ for (int V = -(n - 1); V <= m - 1; V++) {

int start_i = (V < 0) ? -V : 0;
int start_j = (V > 0) ? V : 0;

int len = (n - start_i < m - start_j) ? n - start_i : m - start_j;

if (len <= 1)
    continue;

double d[len];
int pi[len], pj[len];

/* сохраняем диагональ */
for (int k = 0; k < len; k++) {
    int i = start_i + k;
    int j = start_j + k;
    d[k] = a[i][j];
    pi[k] = i;
    pj[k] = j;
}

int shift = T % len;
if (shift < 0)
```

```

    shift += len;

if (shift == 0)
    continue;

if (debug) {
    printf("[DEBUG] Диагональ V=%d, длина=%d, сдвиг=%d, направление=%s\n", V,
        len, shift, (V % 2 == 0) ? "вправо (чётная)" : "влево (нечётная)");
}

double tmp[len];

if (V % 2 == 0) {
    /* четная вправо */
    for (int k = 0; k < len; k++)
        tmp[(k + shift) % len] = d[k];
} else {
    /* нечетная влево */
    for (int k = 0; k < len; k++)
        tmp[k] = d[(k + shift) % len];
}

/* возвращаем элементы обратно в матрицу */
for (int k = 0; k < len; k++)
    a[pi[k]][pj[k]] = tmp[k];

} }

int main(int argc, char *argv[]) { int manual = 0; int pos = 1;

/* проверка аргументов командной строки */ if (argc < 3) { fprintf(stderr, "Использование: %s [-m]
строки столбцы\n", argv[0]); fprintf(stderr, "-m ручной ввод элементов матрицы\n"); return 1; }

/* проверка флага -m */ if (strcmp(argv[pos], "-m") == 0) { manual = 1; pos++;

if (argc - pos < 2) {
    fprintf(stderr, "Ошибка: отсутствуют обязательные аргументы после -m\n");
    return 1;
}

}

/* проверка неизвестных опций */ for (int i = 1; i < argc; i++) { if (argv[i][0] == '-' && strcmp(argv[i], "-
m") != 0) { fprintf(stderr, "Ошибка: опция '%s' не поддерживается\n", argv[i]); return 1; } }

if (argc - pos != 2) { if (argc - pos > 2) { fprintf(stderr, "Ошибка: лишние аргументы\n"); } else {
fprintf(stderr, "Ошибка: недостаточно аргументов\n"); } return 1; }

int n, m; if (!parse_int(argv[pos], &n) || !parse_int(argv[pos + 1], &m)) { fprintf(stderr, "Ошибка:
некорректные размеры матрицы\n"); return 1; }

/* матрица */ double a[n][m];

```

```

/* заполнение матрицы */ if (manual) { printf("Введите элементы матрицы %dx%d:\n", n, m); for (int i = 0; i < n; i++) { for (int j = 0; j < m; j++) { if (!read_double(&a[i][j])) { fprintf(stderr, "Ошибка: некорректный ввод числа\n"); return 1; } } } } else { srand((unsigned)time(NULL)); for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) a[i][j] = (rand() % 10000) / 100.0; }

/* ввод T */ printf("Введите T: "); char buf[256]; if (!fgets(buf, sizeof(buf), stdin)) { fprintf(stderr, "Ошибка: некорректный ввод T\n"); return 1; }

char *endptr; errno = 0; long t = strtol(buf, &endptr, 10);

if (endptr == buf) { fprintf(stderr, "Ошибка: '%s' не является числом\n", buf); return 1; }

/* переполнение */ if (errno == ERANGE) { fprintf(stderr, "Ошибка: значение T выходит за диапазон\n"); return 1; }

/* после числа нет лишних символов */ while (*endptr == ' ' || *endptr == '\t') endptr++; if (*endptr != '\0' && *endptr != '\n') { fprintf(stderr, "Ошибка: некорректный формат T\n"); return 1; }

int T = (int)t;

printf("\nИсходная матрица:\n"); print_matrix(n, m, a);

shift_diagonals(n, m, a, T);

printf("\nРезультат:\n"); print_matrix(n, m, a);

return 0; }

```

```

1  #include <errno.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <time.h>
6
7  /* чтение целого числа из строки */
8  int parse_int(const char *s, int *x) {
9      char *endptr;
10     errno = 0;
11     long val = strtol(s, &endptr, 10);
12
13     if (endptr == s || *endptr != '\0' || errno != 0)
14         return 0;
15
16     if (val ≤ 0 || val > 1000)
17         return 0;
18
19     *x = (int)val;
20     return 1;
21 }
22
23 /* чтение числа double */
24 int read_double(double *x) {
25     char buf[256];
26     char *endptr;
27
28     if (!fgets(buf, sizeof(buf), stdin))
29         return 0;
30
31     errno = 0;
32     double val = strtod(buf, &endptr);
33
34     if (endptr == buf || errno != 0)
35         return 0;
36
37     *x = val;
38     return 1;
39 }
40
41 /* вывод матрицы */
42 void print_matrix(int n, int m, double a[n][m]) {
43     for (int i = 0; i < n; i++) {
44         for (int j = 0; j < m; j++)
45             printf("%8.3f ", a[i][j]);
46         printf("\n");
47     }
48 }
49
50 /* циклический сдвиг диагоналей */
51 void shift_diagonals(int n, int m, double a[n][m], int T) {
52     int debug = getenv("LAB2DEBUG") != NULL;
53
54     /* перебор всех диагоналей */
55     for (int V = -(n - 1); V ≤ m - 1; V++) {
56
57         int start_i = (V < 0) ? -V : 0;
58         int start_j = (V > 0) ? V : 0;
59
60         int len = (n - start_i < m - start_j) ? n - start_i : m - start_j;

```

```

61
62     if (len ≤ 1)
63         continue;
64
65     double d[len];
66     int pi[len], pj[len];
67
68     /* сохраняем диагональ */
69     for (int k = 0; k < len; k++) {
70         int i = start_i + k;
71         int j = start_j + k;
72         d[k] = a[i][j];
73         pi[k] = i;
74         pj[k] = j;
75     }
76
77     int shift = T % len;
78     if (shift < 0)
79         shift += len;
80
81     if (shift == 0)
82         continue;
83
84     if (debug) {
85         printf("[DEBUG] Диагональ V=%d, длина=%d, сдвиг=%d, направление=%s\n", V,
86             len, shift, (V % 2 == 0) ? "вправо (чётная)" : "влево (нечётная)");
87     }
88
89     double tmp[len];
90
91     if (V % 2 == 0) {
92         /* четная вправо */
93         for (int k = 0; k < len; k++)
94             tmp[(k + shift) % len] = d[k];
95     } else {
96         /* нечетная влево */
97         for (int k = 0; k < len; k++)
98             tmp[k] = d[(k + shift) % len];
99     }
100
101     /* возвращаем элементы обратно в матрицу */
102     for (int k = 0; k < len; k++)
103         a[pi[k]][pj[k]] = tmp[k];
104 }
105 }
106
107 int main(int argc, char *argv[]) {
108     int manual = 0;
109     int pos = 1;
110
111     /* проверка аргументов командной строки */
112     if (argc < 3) {
113         fprintf(stderr, "Использование: %s [-m] строки столбцы\n", argv[0]);
114         fprintf(stderr, "    -m  ручной ввод элементов матрицы\n");
115         return 1;
116     }
117
118     /* проверка флага -m */
119     if (strcmp(argv[pos], "-m") == 0) {
120         manual = 1;

```

```

121     pos++;
122
123     if (argc - pos < 2) {
124         fprintf(stderr, "Ошибка: отсутствуют обязательные аргументы после -m\n");
125         return 1;
126     }
127 }
128
129 /* проверка неизвестных опций */
130 for (int i = 1; i < argc; i++) {
131     if (argv[i][0] == '-' && strcmp(argv[i], "-m") != 0) {
132         fprintf(stderr, "Ошибка: опция '%s' не поддерживается\n", argv[i]);
133         return 1;
134     }
135 }
136
137 if (argc - pos != 2) {
138     if (argc - pos > 2) {
139         fprintf(stderr, "Ошибка: лишние аргументы\n");
140     } else {
141         fprintf(stderr, "Ошибка: недостаточно аргументов\n");
142     }
143     return 1;
144 }
145
146 int n, m;
147 if (!parse_int(argv[pos], &n) || !parse_int(argv[pos + 1], &m)) {
148     fprintf(stderr, "Ошибка: некорректные размеры матрицы\n");
149     return 1;
150 }
151
152 /* матрица */
153 double a[n][m];
154
155 /* заполнение матрицы */
156 if (manual) {
157     printf("Введите элементы матрицы %dx%d:\n", n, m);
158     for (int i = 0; i < n; i++) {
159         for (int j = 0; j < m; j++) {
160             if (!read_double(&a[i][j])) {
161                 fprintf(stderr, "Ошибка: некорректный ввод числа\n");
162                 return 1;
163             }
164         }
165     }
166 } else {
167     srand((unsigned)time(NULL));
168     for (int i = 0; i < n; i++)
169         for (int j = 0; j < m; j++)
170             a[i][j] = (rand() % 10000) / 100.0;
171 }
172
173 /* ввод T */
174 printf("Введите T: ");
175 char buf[256];
176 if (!fgets(buf, sizeof(buf), stdin)) {
177     fprintf(stderr, "Ошибка: некорректный ввод T\n");
178     return 1;
179 }
180

```

```

181     char *endptr;
182     errno = 0;
183     long t = strtol(buf, &endptr, 10);
184
185     if (endptr == buf) {
186         fprintf(stderr, "Ошибка: '%s' не является числом\n", buf);
187         return 1;
188     }
189     /* переполнение */
190     if (errno == ERANGE) {
191         fprintf(stderr, "Ошибка: значение T выходит за диапазон\n");
192         return 1;
193     }
194     /* после числа нет лишних символов */
195     while (*endptr == ' ' || *endptr == '\t')
196         endptr++;
197     if (*endptr != '\0' && *endptr != '\n') {
198         fprintf(stderr, "Ошибка: некорректный формат T\n");
199         return 1;
200     }
201
202     int T = (int)t;
203
204     printf("\nИсходная матрица:\n");
205     print_matrix(n, m, a);
206
207     shift_diagonals(n, m, a, T);
208
209     printf("\nРезультат:\n");
210     print_matrix(n, m, a);
211
212     return 0;
213 }

```